

Supplementary Material

In the literature [1], methods combining Photoplethysmography (PPG) and Electrocardiogram (ECG) achieves higher Blood Pressure (BP) estimation performance than methods based only on PPG, considering the clinical standards defined by the Association for the Advancement of Medical Instrumentation (AAMI) [2] and the British Hypertension Society (BHS) [3]. This is because the combination of ECG with PPG provides richer information about the haemodynamics of a subject (e.g., the pulse arrival time). Therefore, deep-neural-networks (DNN) for BP estimation can exploit this additional information through a multimodal architecture that can increase BP estimation performance against AAMI and BHS standards. However, adding the second input modality requires larger DNN architectures to process the ECG, resulting in both higher memory footprint and computational requirements of the DNN execution. Therefore, in this supplementary material, we evaluate the performance-resource-efficiency trade-off of our framework for streaming BP via resource-efficient Continual Learning (CL) with a multimodal approach. Particularly, we provide a quantitative assessment of the continual and clinical performance of resource-efficient CL with PPG and ECG input data modalities, as well as its storage and computational requirements on consumer edge devices. As for the experiments reported in the main manuscript, we used a laptop with a NVIDIA RTX A1000 (4GB) for intensive experiments, while for resource profiling we used a Raspberry Pi 5¹ (16GB) and a Google Pixel 10a² (8GB).

To incorporate the ECG modality within our framework, we expanded the DNN architecture for BP estimation of [4] (FewShotBP). The FewShotBP architecture is made of a convolutional neural network (CNN) followed by a Gated Recurrent Unit (GRU) for PPG feature extraction and 2 fully-connected layers with dropout for Systolic and Diastolic BP (SBP/DBP) regression. We added a second stack of CNN + GRU for ECG feature extraction alongside that for PPG, while also adding a third layer to the prediction head. Without these expansions of FewShoBP, we could not obtain comparable or better clinical performances than the PPG-only version. We report in Table I a breakdown³ of the model parameters of the PPG and PPG+ECG versions, setup with embedding sizes 128 and 256 (128 for PPG and 128 for ECG) respectively. From Table I, we can observe that FewShotBP with PPG and ECG as input data modalities requires more parameters. This is why evaluating our framework with extensive experiments

TABLE I
NUMBER OF PARAMETERS FOR THE FEWSHOTBP ARCHITECTURE WITH PPG AND PPG+ECG INPUT DATA MODALITIES.

DNN Module	PPG	PPG+ECG
Feature extractor	193,384 k	386,768 k
Prediction Head	8,451 k	41,347 k

over multiple seeds can be computationally intensive, therefore for this supplementary material we will present results with a single seed only (42). For the same reason, we reproduced only the main experiments of our framework and only with one CL algorithm. We chose the feature replay CL algorithm since it achieved the best results against other CL algorithms. We reproduced our framework with PPG and ECG using the same dataset of the main manuscript, namely the PulseDB [5]. Importantly, PulseDB provides ECG, PPG and arterial BP synchronized for both MIMICIII [6] and VitalDB [7] datasets, so we could easily reproduce our framework with both PPG and ECG modalities. We followed the same evaluation configurations and settings for the two stages of our framework, as detailed in the main manuscript. First, we performed the pretraining of multimodal FewShotBP on MIMICIII with Meta-Learning [8]. Then, we evaluated its personalization performance on VitalDB with AAMI/BHS clinical standards, CL metrics, and resource-efficiency metrics as per the main manuscript.

In Table II, we evaluate the impact on continual and clinical performances of adding the ECG data modality. In Table II, we report the personalization results obtained by performing adaptation each time a data stream batch is available for an incremental update (without drift detector thereby without skipping updates). We can observe that adding the ECG modality allowed feature replay to achieve comparable or even better clinical performance (columns ME, STD, and BHS) than the PPG-only version. Particularly, feature replay with PPG and ECG meets clinical standards not only for the gradual shift sets (set number 1) but also for the mixed-shift sets (set number 2). This suggests that ECG can improve the BP estimation performances compared to a PPG-only method, while also providing more robustness to more intense distribution shifts. Instead, we observed that feature replay with PPG+ECG exposed higher forgetting (BWT column) compared to the PPG-only version. This suggests that the larger, multimodal architecture for processing PPG+ECG is more prone to forgetting compared to the smaller, single-modality architecture for processing PPG.

In Tables III and IV, we evaluate the impact on resource

¹<https://www.raspberrypi.com/products/raspberry-pi-5/>

²https://store.google.com/product/pixel_10a

³We employed thop <https://github.com/ultralytics/thop>

TABLE II

FEATURE REPLAY PERSONALIZATION RESULTS (SEED 42) ON VITAL DB FOR CONTINUOUS SBP/DBP ESTIMATION FROM PPG AND PPG+ECG, DIVIDED INTO THE THREE SUBJECT SETS 1/2/3. PARENTHESES INDICATE THE TARGET THRESHOLDS FOR THE CLINICAL STANDARDS FOR BOTH SBP/DBP. THE BUFFER OF FEATURE REPLAY HAS SIZE 64 FOR BOTH PPG AND PPG+ECG.

Set	Algorithm	AE↓	BWT↓	ME↓ (<5 mmHg)	STD↓ (<8 mmHg)	BHS↑ (A)
1	cumulative-mean	8.0 / 4.2	0.3 / 0.2	-0.4 / -0.1	11.5 / 6.2	D / A
	feat.replay (PPG)	4.4 / 2.5	0.8 / 0.4	0.7 / 0.4	5.8 / 3.5	A / A
	feat.replay (PPG+ECG)	4.8 / 2.6	1.7 / 0.9	0.6 / 0.4	5.6 / 3.3	A / A
2	cumulative-mean	10.3 / 5.3	0.3 / 0.2	-1.5 / -0.6	14.1 / 7.4	D / B
	feat.replay (PPG)	6.7 / 3.8	2.2 / 1.4	0.5 / 0.3	7.9 / 4.4	B / A
	feat.replay (PPG+ECG)	6.2 / 3.4	2.7 / 1.4	0.5 / 0.3	7.4 / 4.0	A / A
3	cumulative-mean	11.4 / 5.9	0.3 / 0.2	-1.4 / -0.2	15.6 / 8.3	D / B
	feat.replay (PPG)	7.6 / 4.4	2.6 / 1.7	0.3 / 0.3	9.6 / 5.3	C / A
	feat.replay (PPG+ECG)	7.3 / 4.3	3.5 / 2.2	0.3 / 0.3	9.2 / 5.1	B / A

TABLE III

ABLATION STUDY ON MMD, OVER SMALLER EMBEDDING (256/64/32) AND BUFFER (64/32/16) SIZES. FOR CONCISENESS, WE REPORT ONLY THE SBP RESULTS.

Emb.	ME↓	STD↓	BHS↑	Skip%↑
256	0.7 / 0.7 / 0.6	5.8 / 5.8 / 6.2	A / A / A	24.3 / 27.9 / 37.8
64	1.8 / 1.9 / 2.0	6.1 / 6.2 / 6.5	A / A / A	28.5 / 32.5 / 41.9
32	4.8 / 4.8 / 4.4	7.7 / 7.7 / 8.2	C / C / C	39.5 / 43.4 / 56.6

efficiency of adding the ECG data modality. We focus on feature replay with the MMD detector [9] to assess the trade-off between the frequency of skipped updates and clinical performance of PPG+ECG compared to the always-on baseline. Although feature replay with PPG+ECG could meet the clinical standard also for the mixed-shifts set, we discuss the trade-off focusing on the gradual-shifts set to be consistent with the main manuscript experiments. To define how much we can increase the number of skipped updates of feature replay with MMD, we perform an ablation study on the buffer size as number of samples and on the embedding dimension of FewShotBP (Table III). Similar to the results reported in the main manuscript, clinical performance deteriorates, while skipped update fraction increases, with smaller embedding or buffer sizes. Particularly, with smaller buffers, MMD skips more updates, suggesting that MMD detects less drifts when its reference distribution is estimated from small buffer sizes. We could maintain clinical compliance while pushing embedding size to 64 and buffer size to 16 while increasing the skipped update fraction to 41.9%. Therefore, we proceed by profiling the communication, computational, and storage requirements of feature replay with MMD, embedding size 32 and buffer size 16, while adapting to gradual shifts on the Raspberry Pi 5 and the Google Pixel 10a⁴.

We estimated the communication costs of our framework starting from the bytes of the multimodal FewShotBP architecture and one PPG+ECG sample (10s @ 125Hz = 1250 floating-point values times 2 modalities = 9.77kB). We estimated the bytes of the feature extractor θ and the prediction head ψ with thop⁵, obtaining 100.25kB and 10.39kB for θ and ψ ,

⁴Profiling on Pixel 10a was obtained while running other user applications, to reflect realistic on-device conditions.

⁵<https://github.com/ultralytics/thop>

TABLE IV

RESOURCE PROFILING OF FEATURE REPLAY WITH MMD, EMBEDDING SIZE 32 AND BUFFER SIZE 16.

Metric	Raspberry Pi 5	Google Pixel 10a
<i>Estimated Communication Requirements</i>		
Setting A	110.6 kB	110.6 kB
Setting B	2.9 MB	2.9 MB
<i>Profiled Comput. Requirements ~ Update Latency Breakdown (ms)</i>		
feat. extraction	59.5 ± 0.7	696.8 ± 17.1
BP prediction	0.9 ± 0.0	3.8 ± 0.1
drift detection	4.5 ± 0.1	22.4 ± 0.5
head adapt.	22.8 ± 5.8	60.1 ± 15.9
detector reinit.	248.6 ± 66.0	93.8 ± 26.1
<i>Cumulative Latency for 88 subj. (min)</i>		
MMD	43.0 ± 0.0	102.7 ± 0.0
Always-on	18.8 ± 0.0	96.8 ± 0.0
<i>Profiled Memory Requirements ~ Avg. Memory (MB)</i>		
	375.7 ± 0.2	360.5 ± 0.2
<i>Profiled Energy Requirements ~ Avg. Energy (J)</i>		
Update	0.2 ± 0.0	-
Subject	13.0 ± 1.6	-
<i>Profiled Avg. Update Frequency Reduction (%)</i>		
	41.1 ± 14.8	42.6 ± 14.8
<i>Profiled Avg. Annotations required for one subj. (# samples)</i>		
MMD	169.5 ± 42.5	165.2 ± 42.6
Always-on	288.0 ± 0.0	288.0 ± 0.0

respectively. In both personalization settings A/B, the edge devices spend 110.64kB for the one-time communication cost of downloading multimodal FewShotBP, which is the sum of θ and ψ bytes. For personalization setting B, we estimate that the edge devices would devote an additional 39.08kB (4×9.77 kB, the size of x_{t_i}) of communication costs for collecting the streamed PPG batch at each t_i (72 time steps for the “Gradual-Shifts” set), totalling 2.86MB (72×39.08 kB + 110.64kB). To profile the computational requirements of feature replay with MMD, we used the python library *time*⁶ to read the time in seconds required by feature extraction with θ , BP values prediction with ψ_{t_i} , drift detection with ϕ_{t_i} , and adaptation to x_{t_i} (this latter is divided into the update of ψ_{t_i} and the reinitialization of ϕ_{t_i}). From Table IV, we can observe a similar pattern to the profiling results for the PPG-only version of our framework for Raspberry Pi 5. The total CL algorithm latency for a single update (<0.4s for Raspberry Pi 5) is domi-

⁶perf_counter() from <https://docs.python.org/3/library/time.html>

nated by the drift detection ϕ_{t_i} reinitialization, which explains the cumulative latency gap between MMD and “Always-on”. Unlike the PPG-only deployment of our framework on the Pixel 10a, feature extraction is particularly slower than drift reinitialization with multimodal FewShotBP, this is why we observed a small cumulative latency gap with “Always-on” compared to Pi 5. As in the main manuscript, we quantify when drift detection with MMD is computationally worthwhile against “Always-on”. Let p denote the empirical update frequency of ϕ_{t_i} (i.e., number of triggered updates divided by the total number of time steps), $\mathcal{L}_{\psi,u}$ the latency to update the prediction head, $\mathcal{L}_{\phi,d}$ the latency of MMD to detect a drift, and $\mathcal{L}_{\phi,r}$ the latency to reinitialize the MMD detector. The per-step cost of feature replay with MMD is $\mathcal{L}_{\phi,d} + p(\mathcal{L}_{\psi,u} + \mathcal{L}_{\phi,r})$, against $\mathcal{L}_{\psi}$ for “Always-on”. Feature replay with MMD is therefore computationally cheaper than “Always-on” whenever $\mathcal{L}_{\phi,d} + p(\mathcal{L}_{\psi,u} + \mathcal{L}_{\phi,r}) < \mathcal{L}_{\psi,u}$. By solving for p and substituting the corresponding latencies from Table IV, we can estimate when MMD detection is worthwhile for both devices. On the Raspberry Pi 5, this trade-off favors feature replay with MMD whenever the frequency of triggered updates stays below $p \sim 6.8\%$. On the Pixel 10a, this trade-off favors feature replay with MMD whenever the frequency of triggered updates stays below $p \sim 24.5\%$. The prediction head update of the multimodal FewShotBP architecture is computationally heavier than the PPG-only version, increasing the value of p on both devices. This suggests that with computationally-intensive updates of ψ_{t_i} , feature replay with MMD is especially preferable than “Always-on”. Overall, given that the subject-specific data stream produces batches every 40s, the CL algorithm latency can also match the data stream speed with multimodal FewShotBP and run personalization before the next batch collection without delaying the BP estimation. To quantify the storage requirements of feature replay with MMD, we used the python library *psutil*⁷ to read the peak memory required by the process that runs personalization with the feature replay algorithm, adapting to gradual shifts with the MMD detector. The peak memory usage occurs during the incremental update, when the following components should reside in the edge device memory: reservoir buffer employed by both feature replay and MMD ($64 \times 16 = 4\text{kB}$), DNN weights (110.64kB), DNN activation memory⁸ for θ (10.33MB) and ψ_{t_i} (3.39kB), gradients (10.39kB) and Adam optimizer states (20.77kB) for ψ_{t_i} , and input data (9.77kB). The analytically estimated peak from these components is $\sim 11\text{MB}$ while the measured peak is below 400MB. This difference is attributable to PyTorch and Python runtime. Overall, the measured memory peak is within the memory budgets of the edge devices employed in this study.

To profile the energy requirements of feature replay with MMD, we proceeded as in the main manuscript and focused only on the Raspberry Pi 5 device (set at 2.4 GHz). Therefore, we report in Table IV the average net energy in Joules required

for a single update, and the average net cumulative energy in Joules required to personalize a subject over its full sequence of updates.

Although a latency gap persists between MMD and “Always-on” also when implementing our framework with PPG+ECG, in exchange for the latency overhead of the detector reinitialization, feature replay with MMD requires roughly $\sim 40\%$ less annotated data. This supports that resource-efficient drift-aware updates of feature replay with MMD are more practically useful for edge-based healthcare systems than “Always-on”, also when implementing our framework with PPG+ECG.

REFERENCES

- [1] S. Maqsood, S. Xu, S. Tran, S. Garg, M. Springer, M. Karunanithi, and R. Mohawesh, “A survey: From shallow to deep machine learning approaches for blood pressure estimation using biosensors,” *Expert Systems with Applications*, vol. 197, p. 116788, 7 2022.
- [2] W. B. White, A. S. Berson, C. Robbins, M. J. Jamieson, L. M. Prisant, E. Roccella, and S. G. Sheps, “National standard for measurement of resting and ambulatory blood pressures with automated sphygmomanometers,” *Hypertension*, vol. 21, pp. 504–509, 4 1993, doi: 10.1161/01.HYP.21.4.504.
- [3] B. Williams, N. R. Poulter, M. J. Brown, M. Davis, G. T. McInnes, J. F. Potter, P. S. Sever, and S. M. Thom, “British hypertension society guidelines for hypertension management 2004 (bhs-iv): summary,” *BMJ*, vol. 328, pp. 634–640, 2004.
- [4] F. Fan, Y. Gu, J. Shen, F. Dong, and Y. Chen, “Fewshotbp: Towards personalized ubiquitous continuous blood pressure measurement,” *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 7, 9 2023.
- [5] W. Wang, P. Mohseni, K. L. Kilgore, and L. Najafizadeh, “Pulsedb: A large, cleaned dataset based on mimic-iii and vitaldb for benchmarking cuff-less blood pressure estimation methods,” *Frontiers in Digital Health*, vol. Volume 4 - 2022, 2023.
- [6] A. E. W. Johnson, T. J. Pollard, L. Shen, L. wei H Lehman, M. Feng, M. Ghassemi, B. Moody, P. Szolovits, L. A. Celi, and R. G. Mark, “Mimic-iii, a freely accessible critical care database,” *Scientific Data*, vol. 3, p. 160035, 2016.
- [7] H.-C. Lee, Y. Park, S. B. Yoon, S. M. Yang, D. Park, and C.-W. Jung, “Vitaldb, a high-fidelity multi-parameter vital signs database in surgical patients,” *Scientific Data*, vol. 9, p. 279, 2022.
- [8] A. Raghu, M. Raghu, S. Bengio, and O. Vinyals, “Rapid learning or feature reuse? towards understanding the effectiveness of maml,” *CoRR*, vol. abs/1909.09157, 2019.
- [9] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.

⁷<https://pypi.org/project/psutil/>

⁸https://huggingface.co/blog/train_memory